

PF00 -Actividad final Tipo R - DTPR



Arquitectura de Computadoras

Alumno: Byron Javier González Medrano

Alumno: Fatima Lisette Hernandez Barajas

Alumno: Juan David Martinez Santana

Alumno: Mariana Soto Almaguer

Alumno: María Guadalupe Esparza Flores

Equipo: ALU-Cinógenos

Sección: D12

Profesor: Jorge Ernesto López Arce Delgado

Introducción

El diseño de procesadores es una de las áreas fundamentales en la arquitectura de computadoras. Por lo que una vez comprendidos los elementos básicos como la memoria, la Unidad Aritmético-Lógica (ALU) y el banco de registros, es posible integrarlos para construir un datapath funcional capaz de ejecutar instrucciones reales.

El Datapath Tipo R es una versión simplificada del procesador MIPS que permite ejecutar instrucciones aritméticas y lógicas del formato R.

En el contexto de la materia de Arquitectura de Computadoras, este módulo representa la integración de los conceptos previos sobre ALU, banco de registros y memoria, añadiendo los módulos de control necesarios para la decodificación y ejecución de instrucciones. La unidad de control interpreta los campos de la instrucción y genera las señales necesarias para coordinar todos los componentes del sistema.

A continuación se procede a explicar cada uno de los nuevos conceptos necesarios para la realización de esta actividad.

instrucciones tipo R, con énfasis en explicar la instrucción SLT.

Para darle instrucciones a una computadora se necesita hablar en su lenguaje, lo equivalente para una palabra en el lenguaje de una computadora es una instrucción, estas se encuentran almacenadas en un conjunto de comandos dentro de un set de instrucciones.

Los microprocesadores mips utilizan palabras de 32 bits para una instrucción.

Existen diferentes tipos de instrucciones, entre ellas podemos encontrar las instrucciones tipo R (registros), tipo I (inmediatas), y tipo J (saltos).

Las instrucciones tipo R en la arquitectura MIPS son aquellas en las que se opera con registros, esto significa que no acceden a la memoria para obtener los datos de entrada.

Todas las instrucciones tienen un ancho de palabra de 32 bits, su formato está dividido en 6 campos como se muestra a continuación:

- op: es el código de operación
- rs: es el primer operando
- rt: es el segundo operando
- rd: registro destino
- shamt: desplazamiento
- funct: indica la operación específica.

Es importante destacar que el campo Opcode para todas las instrucciones tipo R es siempre cero (000000), lo que indica al procesador que debe consultar el campo Funct para determinar la operación específica a realizar. Este esquema permite definir hasta 64 operaciones diferentes dentro del formato tipo R.

Estructura de una instrucción tipo R.

Campo	Cantidad de bits	Descripción
op	6	Código de operación
rs	5	Registro del primer operando fuente
rt	5	Registro del segundo operando fuente
rd	5	Registro destino
shamt	5	Desplazamiento
funct	6	Código de función

Tabla 1.1 – (Estructura de instrucción de tipo R)

Entre las instrucciones de tipo R podemos encontrar add (suma), sub (resta), and, or, y slt (comparación menor que). De esta última se explica un poco más a continuación:

La instrucción SLT es una de las más importantes dentro de las instrucciones tipo R, su función es comparar dos valores almacenados en los registros y establecer un valor booleano en el registro destino según el resultado de comparación.

Su sintaxis en ensamblador es:

SLT \$rd, 4rs, \$rt

donde:

\$rs : Primer registro operando (registro fuente 1)

\$rt : Segundo registro operando (registro fuente 2)

\$rd : Registro destino donde se guarda el resultado (0 o 1)

Esta instrucción es muy importante para la implementación de estructuras condicionales en lenguaje de alto nivel. Por ejemplo, una comparación como if (a < b) en C puede traducirse directamente a una instrucción SLT seguida de una instrucción de salto condicional.

En términos de hardware, la instrucción SLT requiere que la ALU sea capaz de realizar una comparación de menor que entre dos operandos. Esto se logra restando los operandos y analizando el signo del resultado, o mediante un comparador dedicado dentro de la unidad aritmético-lógica.

Una manera de establecer esta instrucción sería utilizando el operador ternario que básicamente funciona como un condicional donde se ejecuta cierta acción si se cumple una condición. La operación se expresa como: $\text{resultado} = (A < B) ? 1 : 0$; lo que significa "Si A es menor que B, asigna 1 al resultado; de lo contrario, asigna 0".

En Verilog, este operador se puede implementar dentro de la unidad aritmética lógica.

Operación ternaria.

Es un mecanismo que permite evaluar una condición y seleccionar entre dos valores posibles en una sola expresión compacta. Su nombre proviene del hecho de que involucra tres operaciones: una condición booleana, un valor para cuando la condición es verdadera, y un valor para cuando es falsa.

su sintaxis es:

$\text{condición} ? \text{valor_si_verdadero} : \text{valor_si_falso}$.

Dentro de verilog y otros lenguajes este operador es muy útil pues permite modelar por ejemplo multiplexores, operaciones condicionales, o selección de caminos de datos de manera directa e intuitiva. En comparación con la estructura if-else , el operador ternario produce un hardware más predecible y generalmente más eficiente en términos de área y velocidad.

Proceso de Compilación.

Gracias al proceso de compilación es que una máquina puede entender lo que escribimos en un lenguaje de alto nivel. Durante este proceso el código fuente se convierte en código máquina por un procesador.

En el contexto de la arquitectura MIPS, este proceso es relevante porque determina cómo las instrucciones en lenguaje de alto nivel se traducen a las instrucciones tipo R que nuestro datapath puede ejecutar.

el proceso completo sigue la siguiente secuencia:

Código fuente ->Análisis -> Código intermedio -> Optimización -> Ensamblador -> Código máquina -> Ejecución.

A continuación se mencionan las etapas del proceso de compilado:

1. Análisi Léxico:

El compilador lee el programa carácter por carácter y lo agrupa en tokens, que son las unidades básicas del lenguaje. En esta etapa se detectan errores como símbolos inválidos y caracteres no reconocidos.

ejemplo:
a = b + c;
se convierte en:
[a] [=] [b] [+] [c] [;]

2. Análisis Sintáctico:

En esta etapa, los tokens se organizan en una estructura jerárquica llamada árbol sintáctico. Aquí se verifica que el programa esté bien escrito según la gramática del lenguaje.

3. Análisis semántico:

Aquí el compilador verifica que el programa tenga sentido lógico, que las variables declaradas utilicen tipos compatibles y las operaciones sean válidas. También se construye la tabla de símbolos que guarda información de variables y registros.

4. Generación de Código Intermedio:

En esta etapa, el código se traduce a una representación intermedia independiente del hardware.

Ejemplo:
f = (g + h) - (i + j);
se convierte en:
t0 = g + h
t1 = i + j
f = t0 - t1

5. Optimización de código:

El compilador mejora el código para hacerlo más rápido, eficiente, y pequeño. Así se logra el uso eficiente de registros y la reducción a accesos de memoria.

6. Generación de código objeto:

Aquí el código intermedio se traduce a ensamblador MIPS, el compilador decide qué registros usar y cómo organizar las instrucciones.

Por ejemplo:
a = b + c;
se convierte en:
add \$0, \$1, \$2

7. Ensamblado:

En esta etapa, el ensamblador convierte el código mips a binario (32 bits).

Por ejemplo:

add \$0, \$1, \$2

se convierte en:

000000 10001 10010 01000 00000 100000

8. Enlazado:

Aquí se unen varios archivos .o, se resuelven funciones externas y se genera el ejecutable final.

Resumen y explicación del video de la clase.

Banco de registros e instrucciones tipo R.

En el video de la clase, el profesor se enfoca en describir y explicar las instrucciones MIPS para arquitecturas tipo RISC, donde todas las instrucciones tienen un formato fijo de 32 bits. Asimismo, se explica cuales son los formatos principales y cómo están organizadas.

Entre los principales tipos de formatos se encuentran los tipo R, tipo I, y tipo J. Estas se organizan en campos como opcode(6 bits), registros fuente rs y rt, registro destino rd (5 bits), el desplazamiento shamt (5 bits), y el campo funct (6 bits).

Sin embargo, en el video de la clase solo se enfoca por ahora en las instrucciones tipo R. Estas operan entre registros y en su mayoría son aritméticas aunque también las hay lógicas.

Una instrucción tipo R utiliza 6 campos: op, rs, rt, rd, shamt, funct. En estas instrucciones se utiliza el símbolo \$ para referirse a los registros del banco de registros.

Para las funciones tipo R su código de operación (opCode) es 000000, mientras que el campo shamt es generalmente 00000 cuando no se utilizan operaciones de desplazamiento. Este campo se usa principalmente en instrucciones de corrimiento, las cuales no se abordan a profundidad en este curso.

En el video se desarrollan varios ejemplos para comprender cómo funciona el formato de instrucciones mnemónicas y su traducción a lenguaje máquina.

Por ejemplo:

add: suma

add \$rd, \$rs, \$rt

Ejemplo:
add \$6, \$0, \$1

Formato:

op	rs	rt	rd	shamt	funct
000000	0000	00001	00110	00000	100000

Tabla 1.2 – (Ejemplo de instrucción de tipo R)

sub: es la instrucción para la resta.

sub \$rd,\$rs,\$rt

Ejemplo:
sub \$6, \$0, \$1

Formato:

op	rs	rt	rd	shamt	funct
000000	00000	00001	00110	00000	100010

Tabla 1.3 – (Estructura de instrucción de tipo R)

Por otra parte, se explica que las instrucciones en arquitecturas RISC tienen un tiempo de ejecución uniforme, y su ejecución se divide en 5 etapas:

1. Obtener la instrucción.
2. Leer los registros necesarios.
3. Realizar la operación en la ALU.
4. Acceso a memoria (cuando es necesario)
5. Escribir el resultado en el Banco de Registros.

Asimismo, se explican las instrucciones tipo I, las cuales incluyen operaciones con valores inmediatos, transferencias de datos y saltos condicionales. Estas instrucciones se utilizan cuando se necesita operar entre un valor almacenado en un registro y un valor inmediato (constante). Su formato es:

Opcode (6 bits) + Rs (5 bits) + Rt (5 bits) + Inmediato/Dirección (16 bits).

y en estas, además de utilizarse el símbolo de registros \$ se utiliza # para referirse a un valor inmediato.

ejemplo: add immediate

addi \$rt, \$rs, #100; \$rt = \$rs + 100

addi \$31, \$16, #100;

y, las instrucciones tipo j son instrucciones de salto, estas son usadas cuando un salto es necesario y es la instrucción que más espacio tiene para el valor inmediato.

su estructura es:

opCode | direccion

Estas instrucciones contienen únicamente el código de operación y la dirección a la que se desea saltar.

Para concluir, el profesor compara las instrucciones MIPS con funciones en lenguajes de programación, donde los mnemónicos representan las operaciones a ejecutar. Además, menciona que es posible implementar un traductor de instrucciones a lenguaje máquina utilizando herramientas como Python y estructuras de datos como diccionarios.

Objetivo General

Diseñar, implementar y validar un Datapath Tipo R funcional basado en la arquitectura MIPS, integrando los módulos de control y procesamiento para garantizar la correcta ejecución de instrucciones aritméticas, lógicas y de comparación.

Objetivos Específicos

- Integrar de manera coordinada los módulos de Unidad de Control, ALUControl, ALU, Banco de Registros, Memoria y Multiplexores para establecer un flujo de datos eficiente.
- Implementar la lógica necesaria para la decodificación y ejecución precisa de las instrucciones ADD, SUB, AND, OR y SLT dentro del procesador.
- Validar el comportamiento dinámico del diseño mediante simulaciones en ModelSim, verificando la correcta interacción entre los componentes y la integridad de los resultados obtenidos en cada operación.
- Sincronizar los componentes de control con las unidades de procesamiento para asegurar que la transferencia de datos entre el Banco de Registros y la ALU se realice conforme a las especificaciones de la arquitectura MIPS.

Desarrollo teórico:

Formato de instrucciones tipo R

Las instrucciones tipo R (o de registro) en la arquitectura MIPS; son aquellas que operan sobre tres registros: dos fuentes y uno destino.

Estas instrucciones poseen un ancho de palabra de 32 bits, los cuales se segmentan en 6 seis campos que dictan la operación a realizar sobre el hardware.

- Opcode (31:26): Código de operación básico para operaciones tipo R [siempre será cero].
- RS (25:21): Primer registro fuente.
- RT (20:16): Segundo registro fuente.
- RD (15:11): Registro destino que recibe el resultado de la operación.
- Shamt (10:6) Cantidad de desplazamiento utilizando operaciones shift [siendo cero en esta ocasión].
- Function (5:0) Código de función que selecciona la variante específica de la operación indicada en el opcode.

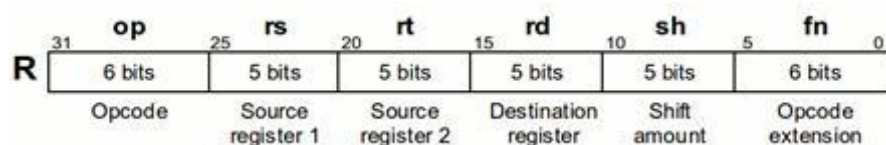


Imagen 1.0 – (Formato de instrucciones de tipo R)

Contador de programa (PC)

El Program counter, es un registro que mantiene almacenada la dirección de la instrucción que está siendo ejecutada actualmente en el sistema. En cada ciclo de reloj, para un datapath de ciclo único, el PC suministra la dirección a la memoria de instrucciones y se incrementa automáticamente en 4 unidades para apuntar a la siguiente palabra de la instrucción.



Imagen 1.1 – (Diagrama de contador de programa)

Banco de registros (Register file)

Este es un componente que contiene una colección de 32 registros generales con 32 bits cada uno. Para instrucciones tipo R, el banco de registros debe ser capaz de realizar dos lecturas simultáneas (especificadas por Rs y Rt) y una de escritura al final del ciclo (especificada por Rd). La escritura es controlada por RegWrite, señal la cual, deberá de estar activa para que los datos se almacenen en el registro destino.

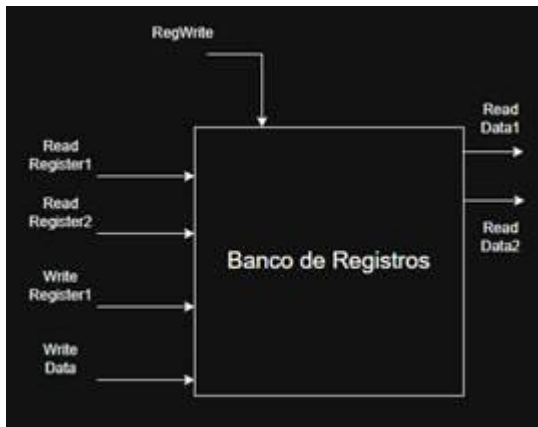


Imagen 1.2 – (Diagrama del banco de registros con sus entradas de lectura, escritura y control, activas)

Unidad aritmético-lógica (ALU)

Es aquel elemento encargado de realizar los cálculos matemáticos y las operaciones lógicas. En el diseño actual, está recibiendo dos operandos de 32 bits provenientes del bando de registros. Es un componente permanentemente combinacional; es decir, que su salida depende exclusivamente de sus entradas actuales en el ciclo.

Para la operación SLT (Set or Less Than), la ALU debe ser capaz de comparar si el valor en RS es menor que el valor en RT. Este proyecto, por medio del operador ternario, se realizará una modificación con el fin de que la eficiencia esté presente.

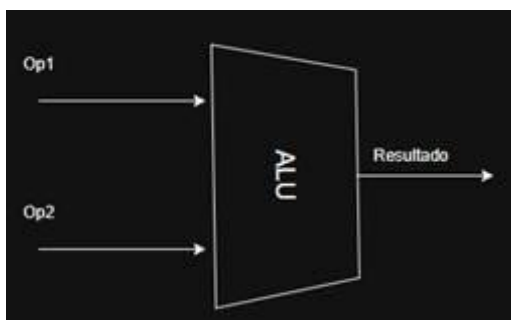


Imagen 1.3 – (Diagrama de la ALU)

Unidad de control (UC) y ALU-Control

Este elemento, ayuda a interpretar el Opcode de la instrucción para generar las señales de selección de los multiplexores y los habilitadores de escritura. Las salidas principales de este módulo incluyen: RegDts, RegWrite y ALUOp.

El ALU-Control, es una unidad de decodificación secundaria que toma la señal ALUOp de la UC y la señal function para formar un campo de 4 bits que indica la operación exacta a realizar (ADD, SUB, AND, OR o SLT)

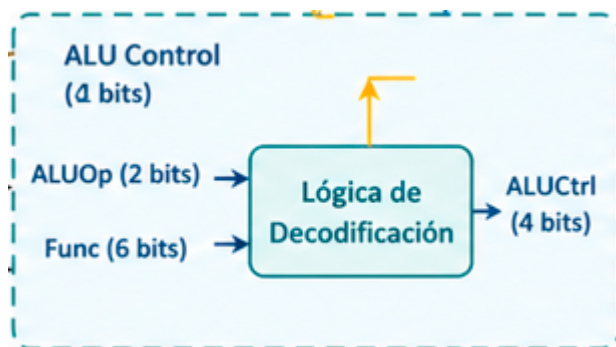


Imagen 1.4 – (Diagrama del funcionamiento de la lógica de decodificación)

Multiplexores (Mux 2:1)

Los multiplexores son dispositivos de selección de datos que permiten que diferentes fuentes compartan un mismo destino. En el datapath R se utiliza un multiplexor para decidir qué campo de la instrucción (RT o RD) define el registro de destino donde se escribirá el resultado, dependiendo de la señal de la Dst.

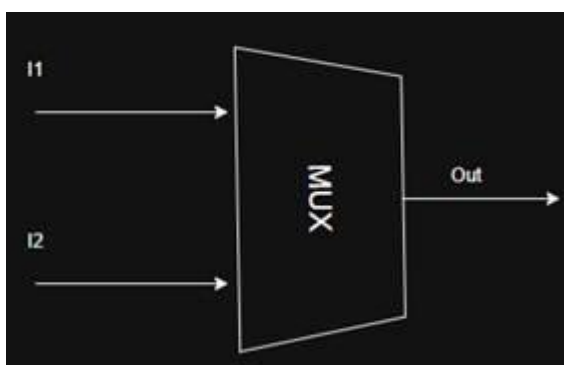


Imagen 1.5 – (Diagrama de multiplexor con las entradas provenientes de la ALU y la memoria implementada)

Memoria

Es un elemento funcional el cual almacena los datos procesados y el código del programa (instrucciones). Para el diseño actual, la memoria de instrucciones suministra la palabra de 32 bits al procesador basándose en la dirección del PC. Aunque las instrucciones de tipo R no escriben directamente en la memoria de datos, el componente para la arquitectura del procesador MIPS, pues, define cómo el procesador obtiene las instrucciones y maneja datos de forma simple, rápida y predecible, lo cual encaja con la filosofía de dicha arquitectura; pues, al estar basada en RISC, busca eficiencia en sus instrucciones.

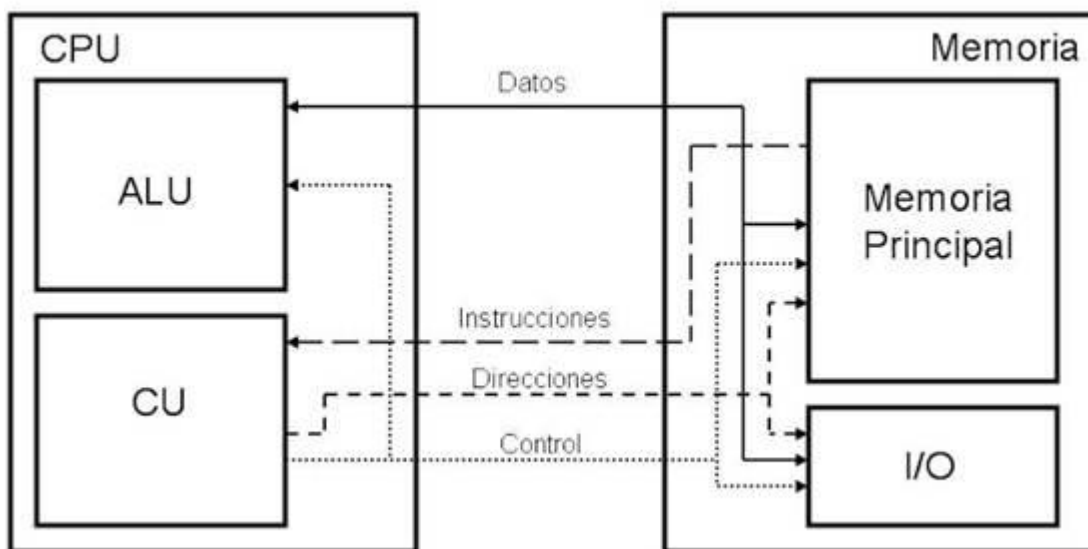


Imagen 1.6 – (Diagrama representativo de cómo funciona el contacto con la memoria para el manejo de instrucciones y proporción de datos)

Desarrollo práctico

Para realizar pruebas de caja blanca/pruebas unitarias de cada módulo, empezamos con el **banco de registros**.

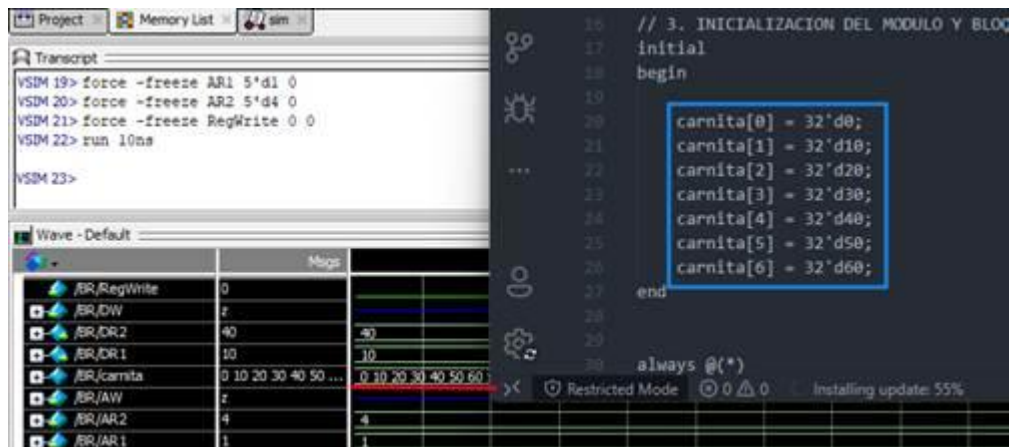


Imagen 1.7 – (Simulación del banco de registros)

Descripción: Los registros inicializados en el programa del banco de registros (recuadro azul), al iniciar la simulación y dar un tiempo de ejecución, se pueden ver reflejados en el registro (línea roja)

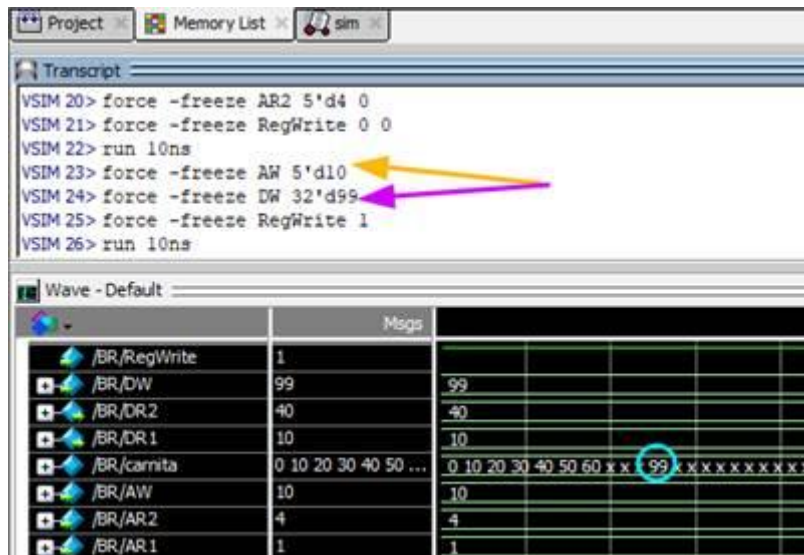


Imagen 1.8 – (Simulación del banco de registros)

Descripción: Al habilitar el valor de 1 en el RegWrite, indicar el registro en el cual almacenar y el valor, se puede ver reflejado este mismo.

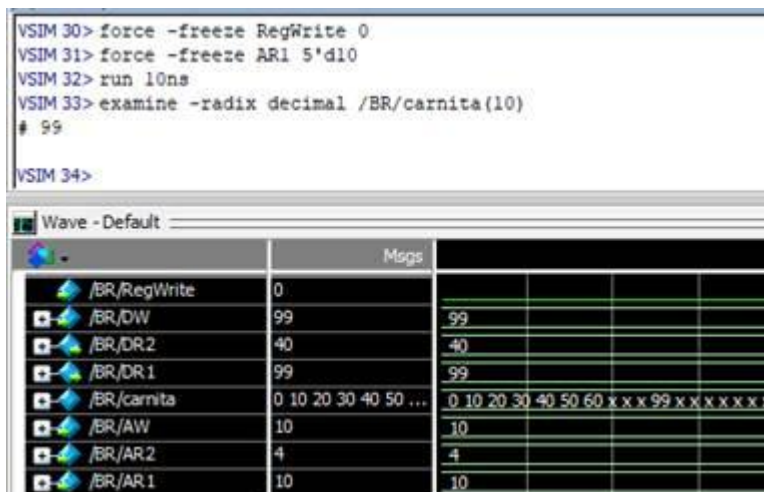


Imagen 1.9 – (Simulación del módulo BR)

Descripción: Al habilitar la lectura, se puede verificar que, en efecto, el valor de 99 se ha guardado correctamente en el registro 10.

A continuación, se realizaron pruebas al módulo de la **unidad de control**, vaticinando como resultado, que cada tipo de instrucción genere las señales correctas.

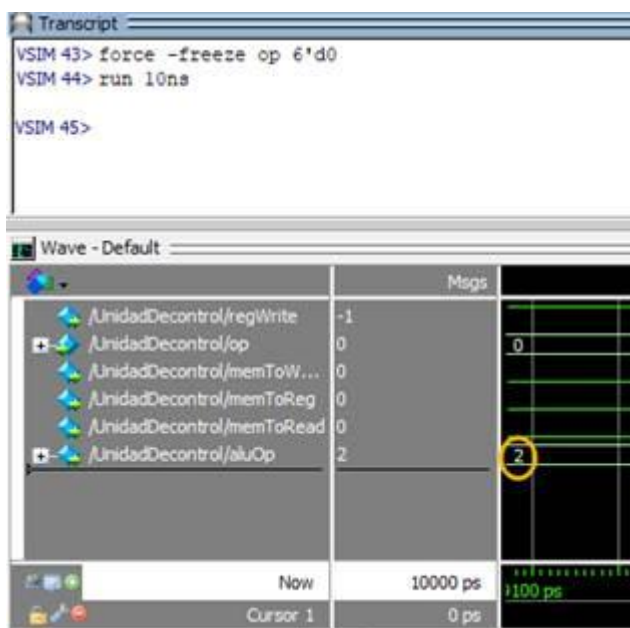


Imagen 2.0 – (Simulación de la unidad de control)

Descripción: Se simula que el procesador acaba de mandar una instrucción tipo R, cayendo en el caso reservado para dicha instrucción configurado de la siguiente manera:

```

always @* begin
  case(op)
    6'b000000: begin // Tipo-R
      memToReg = 1'b0;
      memToRead = 1'b0;
      memToWrite = 1'b0;
      aluOp = 3'b010;
      regWrite = 1'b1;
    end
  end
end

```

Imagen 2.1 – (Bloque de código que evalúa la entrada del op)

Donde: el único valor que cambia sería el aluOp, que en la simulación correctamente puede verse dicho cambio.

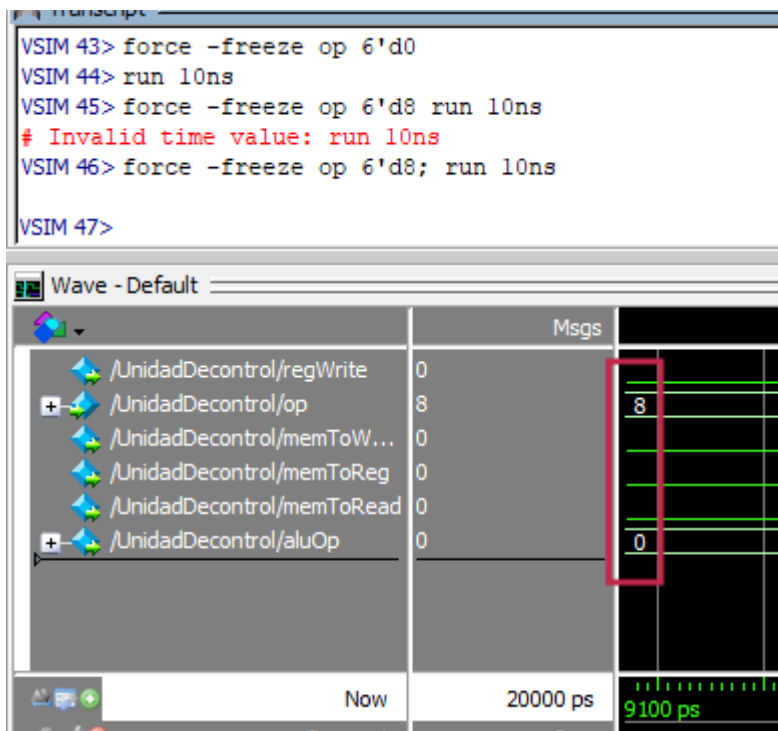


Imagen 2.2 – (Simulación de la unidad de control)

Descripción: Se fuerza un valor fuera del rango del case, obligándolo a caer en el default, donde, como se ve en la imagen 2.1, todo queda igualado a 0.

Para las pruebas del módulo del **ALU control**, el cual es el encargado de recibir la orden general de la unidad de control y decodificarla para dependiendo de ella, se realizará una operación u otra.

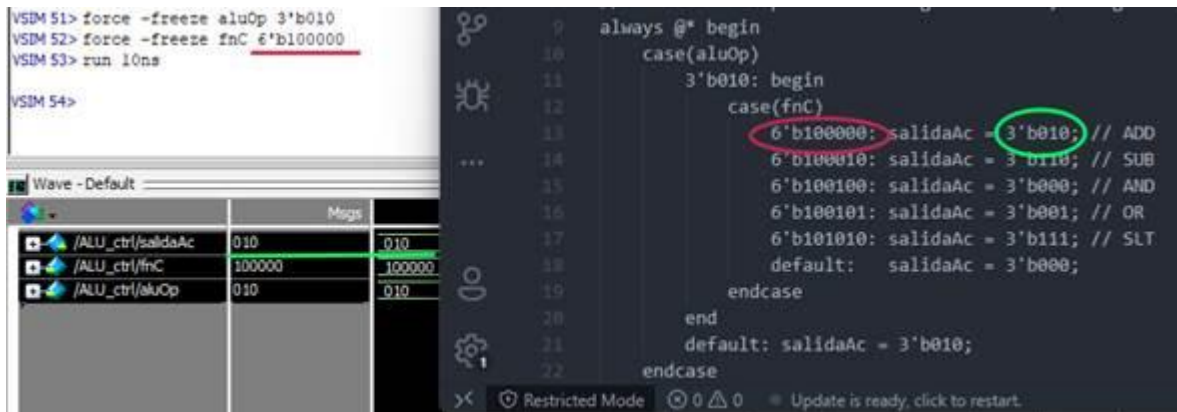


Imagen 2.3 – (Simulación de la ALU control)

Descripción: De entrada se utiliza para evaluar el identificador de instrucciones tipo R (010), el cual, al evaluarse, entra a la bifurcación y por medio del valor designado de la fnC, realizará una u otra operación. Donde en el primer caso, se utilizó la suma el cual el identificador (circulo coloro tinto).

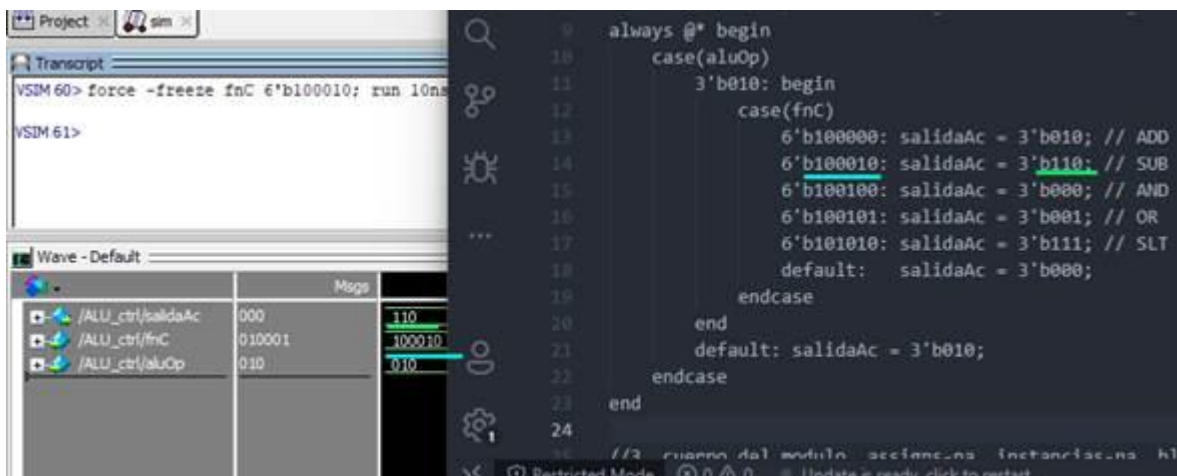


Imagen 2.4 – (Simulación de la ALU control)

Descripción: Se cambia la entrada fnC y se fuerza el código que define la operación de resta, al entrar a la bifurcación, lo identifica y dispara la salida marcada en verde.

Nota: La entrada, el usar -freeze, se queda “congelada” hasta que no le forcemos otro valor, en este caso, el valor de aluOp, se queda en 010, siendo el de la instrucción tipo R.

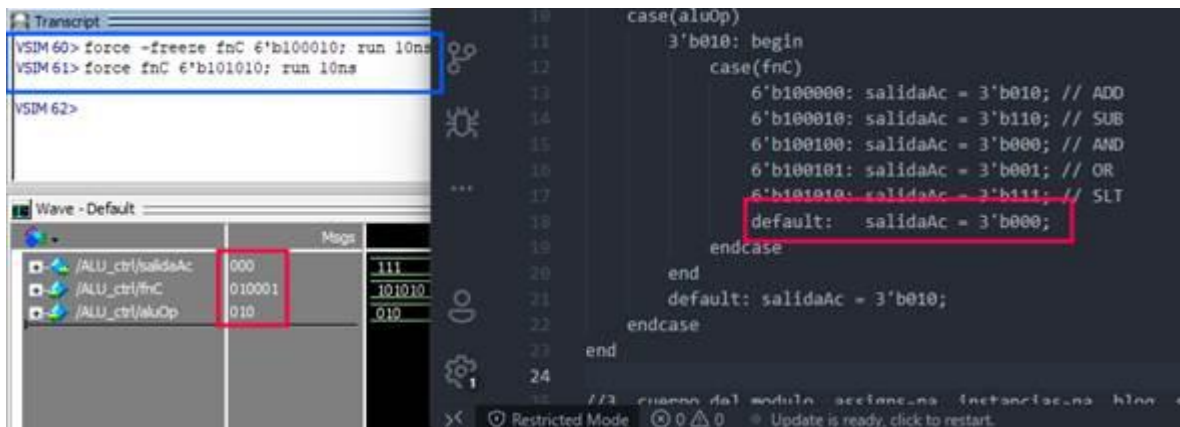


Imagen 2.5 – (Simulación de la ALU control)

Descripción: Al forzar la entrada al default por parte del fnC, se muestran las salidas definidas por este mismo.

Cómo consiguiente y siendo el módulo posterior al anterior, se realizará la prueba unitaria a la **ALU**, la cual se “hardcodeará” el resultado arrojado por el módulo ALU control, para hacer pruebas.

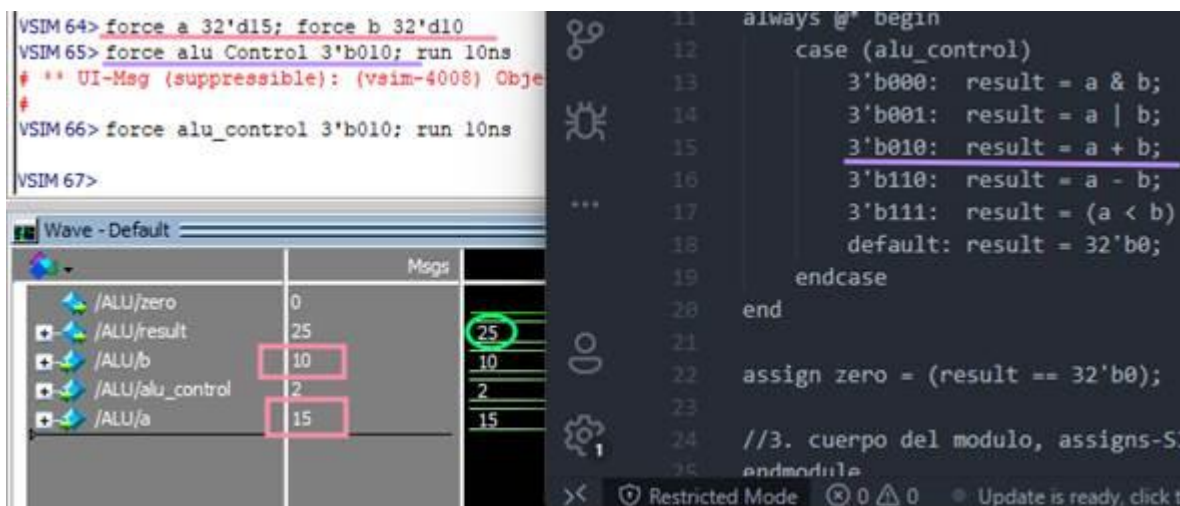


Imagen 2.6 – (Simulación de la ALU)

Descripción: Se fuerzan los valores en la entrada de la ALU siendo a=15 y b=10, y poniendo el código que mandará la unidad anterior (ALU control) de suma (010) para dentro de la bifurcación, seleccionar la misma y ejecutarla dando como resultado 25.

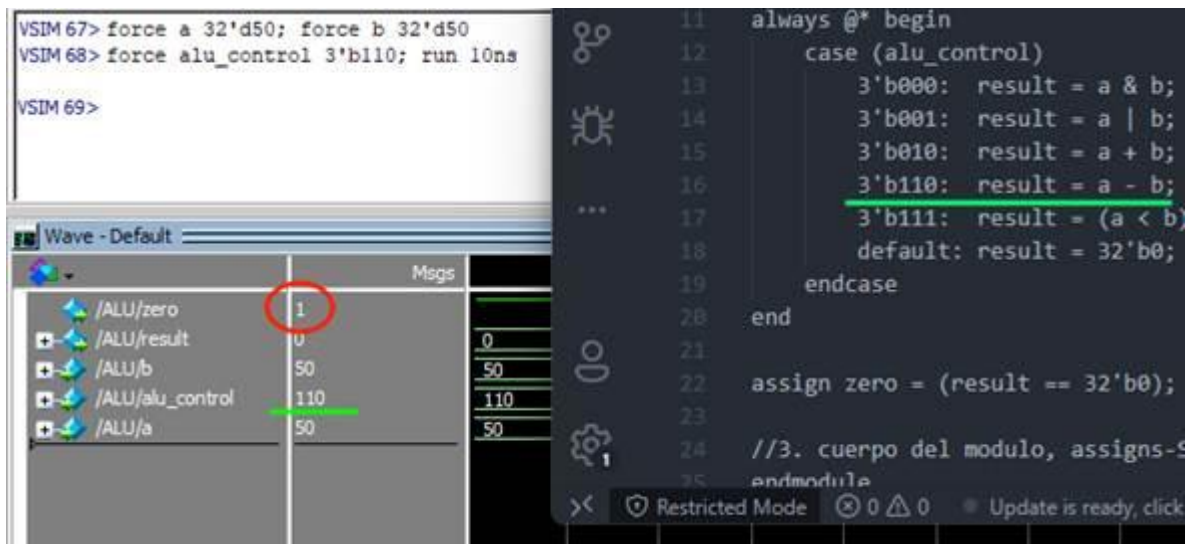


Imagen 2.7 – (Simulación de la ALU)

Descripción: probando con la operación de resta, se simula la llegada de su código (110) desde la ALU control restando las entradas propuestas.

Nota: El valor de la variable zero, puede verse en alto/1, caso contrario de la operación de la imagen 2.6, esto es dado a que es un indicador de validez a manera de bandera que indica si el resultado de la operación es cero, siendo en este caso afirmativo; pues, se restaron dos números iguales.

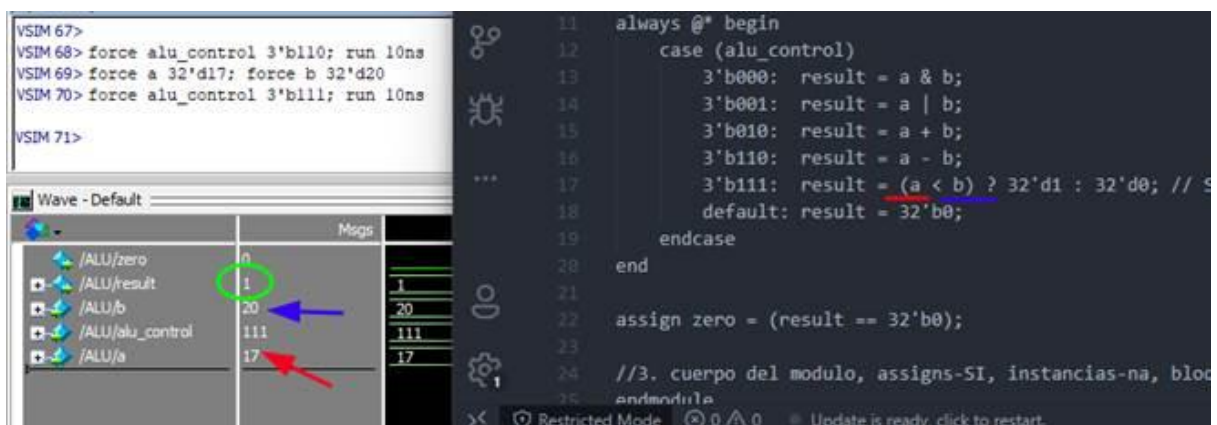


Imagen 2.8 – (Simulación de la ALU)

Descripción: Usando el operador ternario, se verifica la operación SLT, la cual siguiendo la estructura donde se evalúan las dos entradas donde si en caso de que a sea mayor que b, la salida, será 1, caso contrario, será 0.

Siguiendo, con la prueba de la **memoria**, se pueden verificar la escritura y lectura de la dirección.

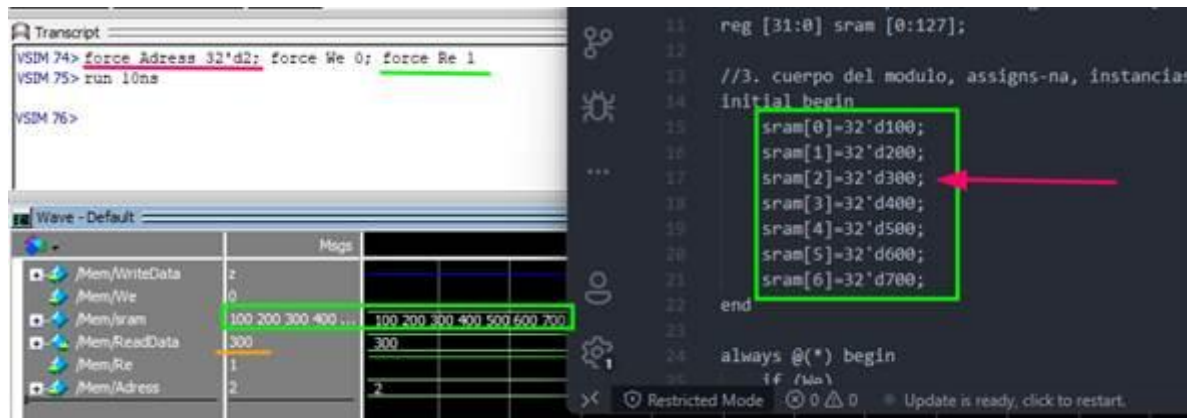


Imagen 2.9 – (Simulación de la memoria)

Descripción: Primeramente, se fuerza un valor en alto para activar la lectura de la dirección que se indica, donde, previamente ya se habría inicializado un valor ([2]:300) y que da como resultado del read data.

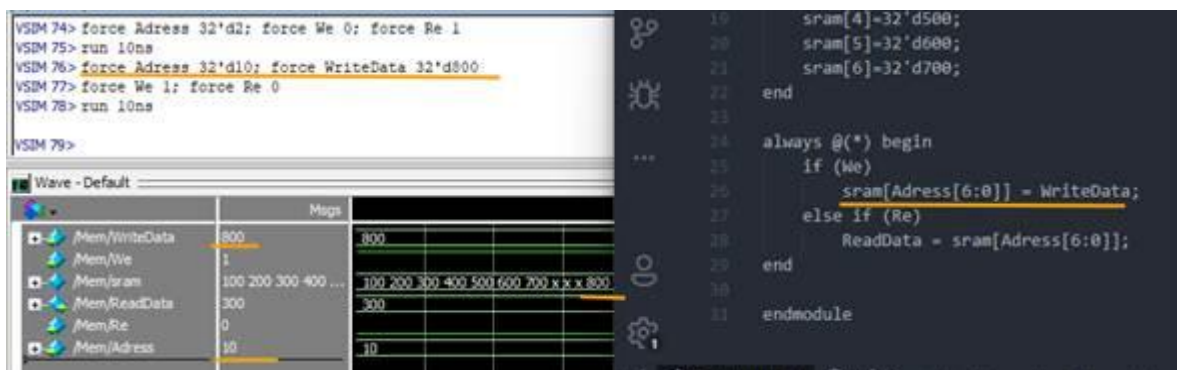


Imagen 3.0 – (Simulación de la memoria)

Descripción: Se fuerzan los valores de escritura, donde, se selecciona la dirección 10 para ponerle el número 800, mismo que se devuelve en el apartado del WriteData.

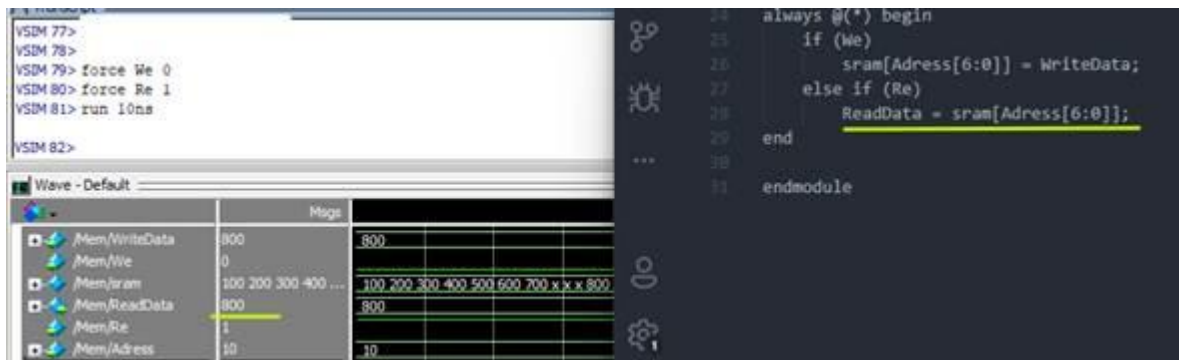


Imagen 3.1 – (Simulación de la memoria)

Descripción: Se fuerza la lectura del mismo elemento que se agregó, llegando al bloque de else if, donde se sustrae el dato leído.

Siguiendo con el módulo final y posterior a este (**MUX**), se comprobará que la salida se seleccione bien según el bit de control.

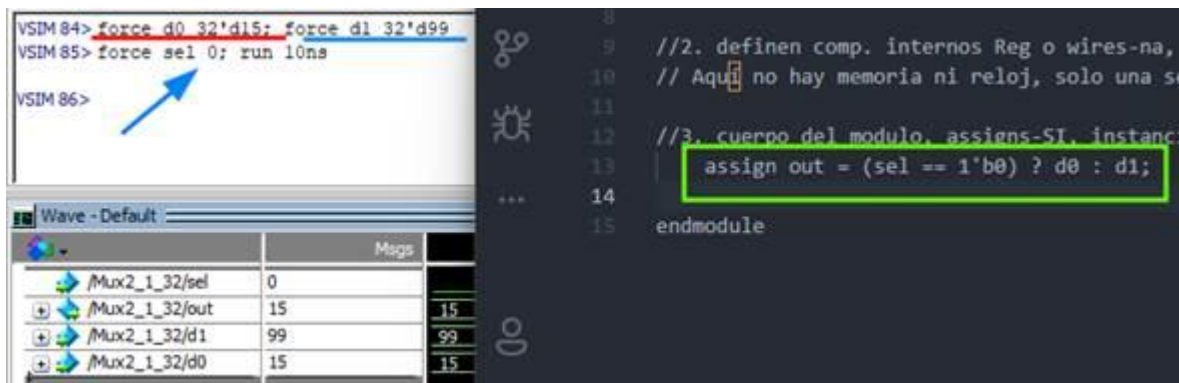


Imagen 3.2 – (Simulación del MUX)

Descripción: Se “inicializan” los valores propuestos por el usuario (15 y 99) y el selector como 0, ignorando el valor de 99 y en la salida poniendo el valor de la entrada 1.

Nota: Se utilizó el operador ternario para ahorrarse líneas de código.

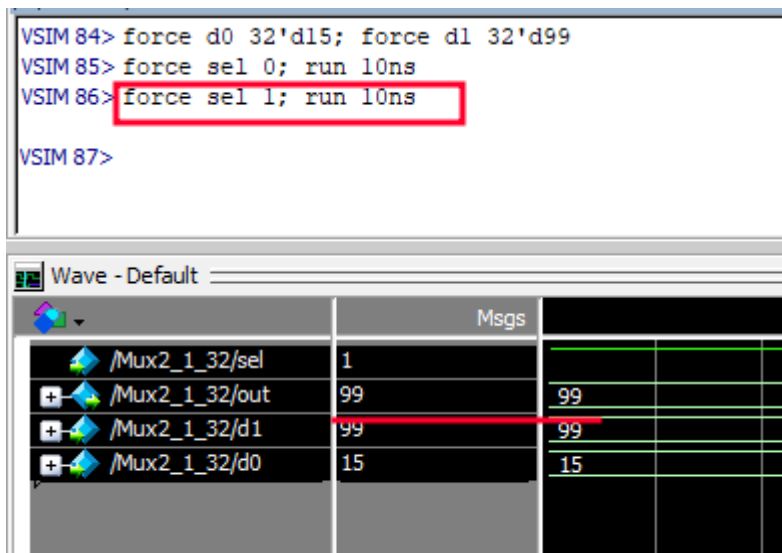


Imagen 3.3 – (Simulación del MUX)

Descripción: Caso contrario al de la imagen 3.2, el selector, se pone en alto, ignorándose el valor de 15 y siguiendo como salida el 99 que anteriormente se ignoraba.

Finalmente, por medio del módulo final, es donde se realizarán todas las conexiones y la conjunción con los diferentes módulos, siendo este el módulo principal (**DPTR**)

Se probará la instrucción de suma siguiendo la siguiente tabla:

add \$7, \$2, \$3

-	\$2	\$3	\$7	-	-
Op	Rs	Rt	Rd	Shamnt	Funct
000000	00010	00011	00111	00000	100000

Tabla 1.4 – (Ejemplo de instrucción a probar)

Descripción: Se utilizan los valores inicializados en el begin del banco de registros [10 y 20] respectivamente.

Nota: Para hacer más “legible” la instrucción, se traducirá a hexadecimal con el fin de no hacer muy verboso el forcé

Instrucción en hexadecimal:

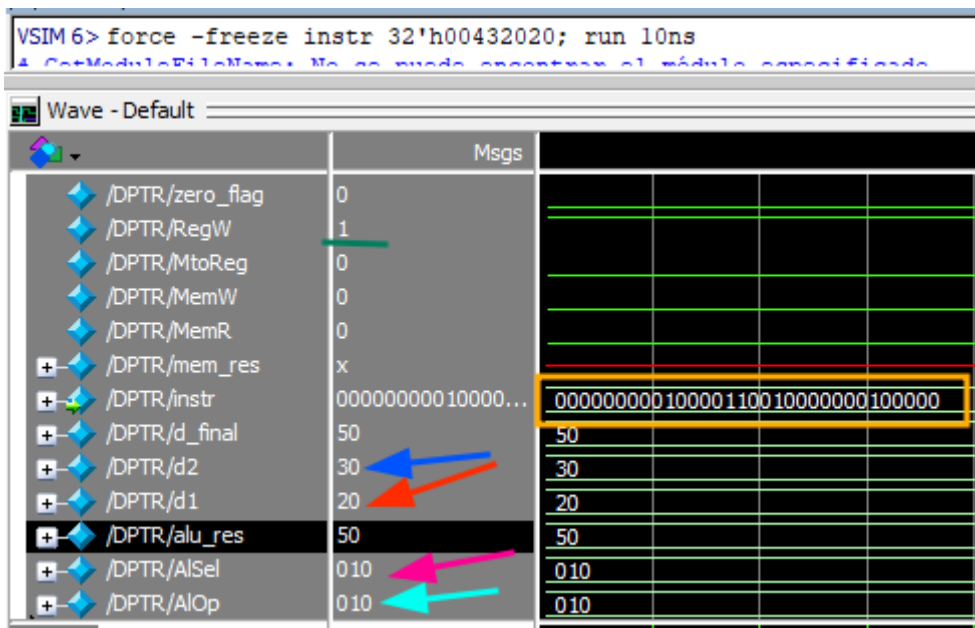


Imagen 3-4 - (Simulación del módulo padre)

Descripción: Se insertó la instrucción mostrada en la tabla 1.4 la cual realiza la operación identificada como 010 (suma) donde tomará los valores del banco de registros [20 y 30] respectivamente para sumarse y que arrojen de resultado su suma (almacenada en alu_res).

Nota: La instrucción se forzó en su equivalente a hexadecimal y se puede verificar que es la misma de la tabla 1.4.

Formato decimal: 000000_00010_00011_00111_00000_100000

Forma hexadecimal: 00432020

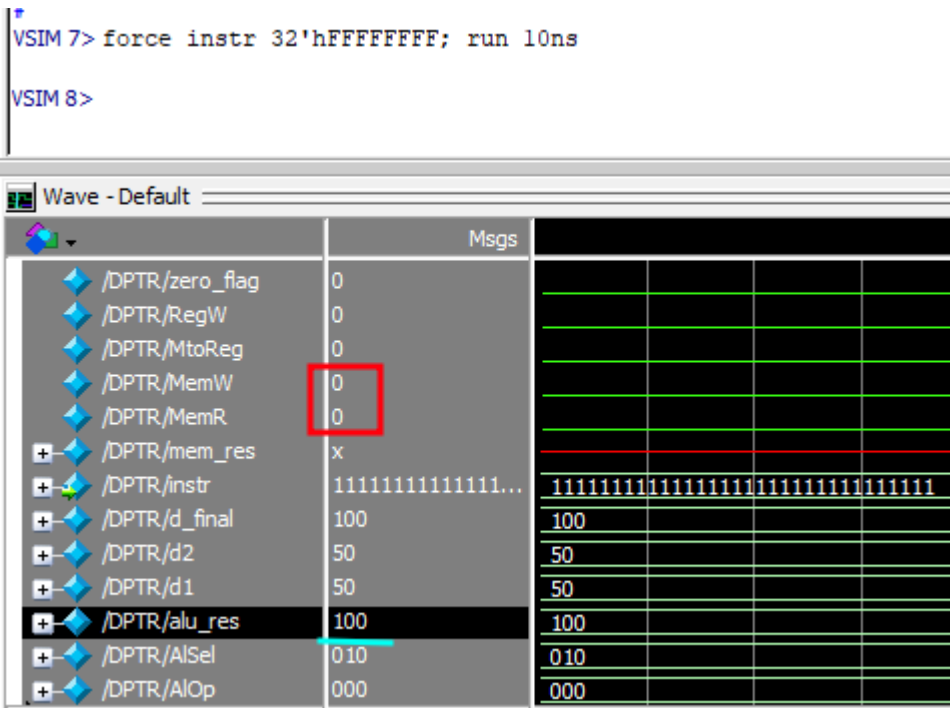


Imagen 3.5 – (Simulación del DPTR)

Descripción: Al solo haber instrucciones de tipo R, forzando el error, en la entrada hexadecimal que se dio, se vuelven cero dados al default, el cual protege las diferentes salidas para evitar que den una operación no válida.

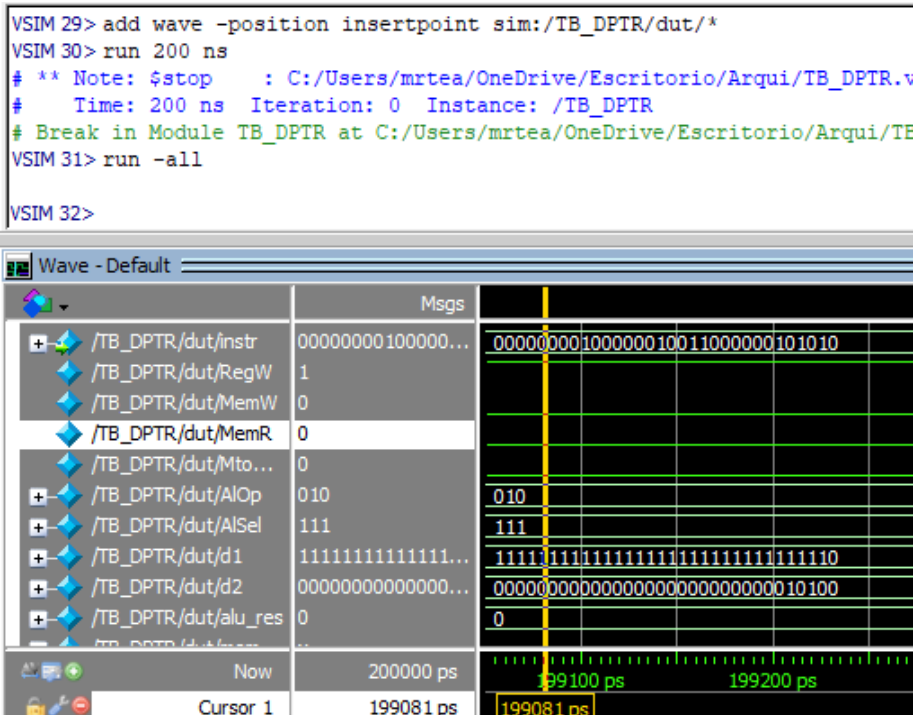


Imagen 3.6 – (Simulación del TB)

			_00010_00110 _00000_1000 10	10
5	AND	and \$1, \$2, \$3	000000_0001 0_00011_0000 1_00000_100 100	R1 = R2 & R3
6	AND	and \$2, \$4, \$5	000000_0010 0_00101_000 10_00000_10 0100	R2 = R4 & R5
7	OR	or \$3, \$1, \$2	000000_0000 1_00010_000 11_00000_100 101	R3 = R1 R2
8	OR	or \$4, \$5, \$6	000000_0010 1_00110_0010 0_00000_100 101	R4 = R5 R6
9	SLT	slt \$5, \$2, \$3	000000_0001 0_00011_0010 1_00000_101 010	R5 = 1
10	SLT	slt \$6, \$4, \$1	000000_0010 0_00001_0011 0_00000_101 010	R6 = 0

Descripción de resultados:

Las pruebas se organizaron en cinco bloques principales:

Bloque de suma (ADD)

Permitió verificar la correcta ejecución de operaciones aritméticas de suma y el almacenamiento de resultados en registros destino.

Bloque de resta (SUB)

Validó el funcionamiento de la operación de resta mediante el código funct = 100010.

Bloque lógico AND

Comprobó el procesamiento bit a bit de operaciones lógicas utilizando funct = 100100.

Bloque lógico OR

Permitió verificar operaciones lógicas OR utilizando funct = 100101.

Bloque de comparación SLT

Se verificó el funcionamiento del operador ternario en la ALU

Conclusión:

La implementación del DataPath Tipo R nos permitió demostrar la integración de los módulos fundamentales de un procesador MIPS simplificado.

En la simulación que hicimos en ModelSim, se logró verificar la correcta ejecución de las instrucciones de prueba, de modo que, los resultados mostraron los valores acorde a las operaciones esperadas, confirmando el total funcionamiento del flujo de datos entre la Unidad de Control, el Banco de Registros, la ALU, ALU Control, multiplexor y la escritura de resultados.

Por otra parte, la instrucción SLT que implementamos mediante el operador ternario nos permitió visualizar la capacidad del sistema para realizar comparaciones condicionales de forma concreta en hardware.

En conclusión, pudimos mostrar la capacidad que tiene un datapath para ejecutar instrucciones mediante la coordinación entre módulos y señales de control, además de servir como una base para procesadores más complejos.

Referencias:

MIPS Assembly/Control Flow Instructions - Wikibooks, open books for an open world. (s. f.).

https://en.wikibooks.org/wiki/MIPS_Assembly/Control_Flow_Instructions

COE1502 - MIPS R2000 Architecture Instruction Formats. (s. f.).

<https://people.cs.pitt.edu/~don/coe1502/Reference/InstructionFormat.html>

instrucciones del lenguaje de las computadoras

chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://www.utm.mx/~merg/AC/pdfs/Unit_2_Part_1.pdf

Scott Moore. (2015, 12 noviembre). IFT201 MIPS Data Path Lecture [Vídeo]. YouTube. <https://www.youtube.com/watch?v=YGSAWqQy9bl>